

National Blood & Organ Donation Network

INDEX

1.Title Page

2.Index

3.Abstract

4.Introduction

5.Methodology

6.Programs

7.Results Screenshots

8.Conclusion

9.References

10.PPT

ABSTRACT

The National Blood & Organ Donation Network is a web-based system designed to streamline donor-patient matching for blood and organ transplants. By integrating multiple data structures and algorithms—such as hashing, decision trees, sorting, and graphs—the system ensures faster, more accurate matching while minimizing delays and improving life-saving outcomes.

INTRODUCTION

Finding compatible blood or organ donors quickly is a major challenge in healthcare. The lack of centralized systems, real-time data, and prioritization mechanisms often results in fatal delays.

This project aims to solve these issues by developing a centralized digital network that connects patients, donors, and hospitals efficiently using intelligent algorithms and data structures.

METHODOLOGY / ARCHITECTURE

The project uses a three-layer architecture with a frontend, backend, and database. The frontend (React.js/Angular) provides dashboards and real-time alerts. The backend (Node.js/Python) handles donor matching, ranking, and routing. Donor and patient data are stored in MongoDB/MySQL using hashing for fast access. Compatibility is checked using a decision tree, donors are ranked with sorting algorithms, and the shortest route is found using graph algorithms like Dijkstra's. The system then sends instant alerts to donors and hospitals.

PROGRAM

```
Classes Debug Untitled1.cpp Untitled2.c
1  #include <stdio.h>
2  #include <string.h>
3  #include <math.h>
4
5  #define MAX 10
6  #define PI 3.14159265358979323846
7  #define EARTH_RADIUS 6371 // in kilometers
8
9  // ===== STRUCTS =====
10 struct Donor {
11     char name[20];
12     char blood[5];
13     char organ[10];
14     int age;
15     char health[10];
16     int available;
17     char location[20];
18 };
19
20 struct Patient {
```

```
19
20 struct Patient {
21     char blood[5];
22     char need[10];
23     int age;
24     char location[20];
25 };
26
27 struct Match {
28     char name[20];
29     float score;
30     float distance;
31     float finalScore;
32 };
33
```

File Edit Search View Project Execute Tools AStyle Window Help

(globals)

Untitled1.cpp Untitled2.c

```
46     return 0;
47 }
48
49 // ===== CITY COORDINATES =====
50 float getLatitude(char city[]) {
51     if (strcmp(city, "Delhi") == 0) return 28.6139;
52     if (strcmp(city, "Mumbai") == 0) return 19.0760;
53     if (strcmp(city, "Bangalore") == 0) return 12.9716;
54     if (strcmp(city, "Chennai") == 0) return 13.0827;
55     if (strcmp(city, "Hyderabad") == 0) return 17.3850;
56     if (strcmp(city, "Kolkata") == 0) return 22.5726;
57     return 0;
58 }
59
60 float getLongitude(char city[]) {
61     if (strcmp(city, "Delhi") == 0) return 77.2090;
62     if (strcmp(city, "Mumbai") == 0) return 72.8777;
63     if (strcmp(city, "Bangalore") == 0) return 77.5946;
64     if (strcmp(city, "Chennai") == 0) return 80.2707;
65     if (strcmp(city, "Hyderabad") == 0) return 78.4867;
66     if (strcmp(city, "Kolkata") == 0) return 88.3639;
67     return 0;
68 }
69
70 // ----- HAUVERSTINE FORMULA -----
```

Compiler Resources Compiler Log Debug Find Results

C:\Users\harsh\Downloads\Code\Untitled2.c - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

(globals)

Untitled1.cpp Untitled2.c

```
70 // ===== HAVERSINE FORMULA =====
71 float toRadians(float degree) {
72     return degree * (PI / 180.0);
73 }
74
75 float distance(char city1[], char city2[]) {
76     float lat1 = getLatitude(city1);
77     float lon1 = getLongitude(city1);
78     float lat2 = getLatitude(city2);
79     float lon2 = getLongitude(city2);
80
81     float dLat = toRadians(lat2 - lat1);
82     float dLon = toRadians(lon2 - lon1);
83
84     float a = sin(dLat / 2) * sin(dLat / 2) +
85             cos(toRadians(lat1)) * cos(toRadians(lat2)) *
86             sin(dLon / 2) * sin(dLon / 2);
87     float c = 2 * atan2(sqrt(a), sqrt(1 - a));
88     return EARTH_RADIUS * c;
89 }
90
91 // ===== SCORE CALCULATION =====
92 float calculateScore(float distance, float radius) {
93     return (1 - distance / radius);
94 }
```

Compiler Resources Compile Log Debug Find Results

Line: 9 Col: 23 Sel: 0 Lines: 205 Length: 6365 Insert Done parsing in 0.031 seconds

6 Search hp M DEV 18:51 29-10-2025

C:\Users\harsh\Downloads\Code\Untitled2.c - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

(globals)

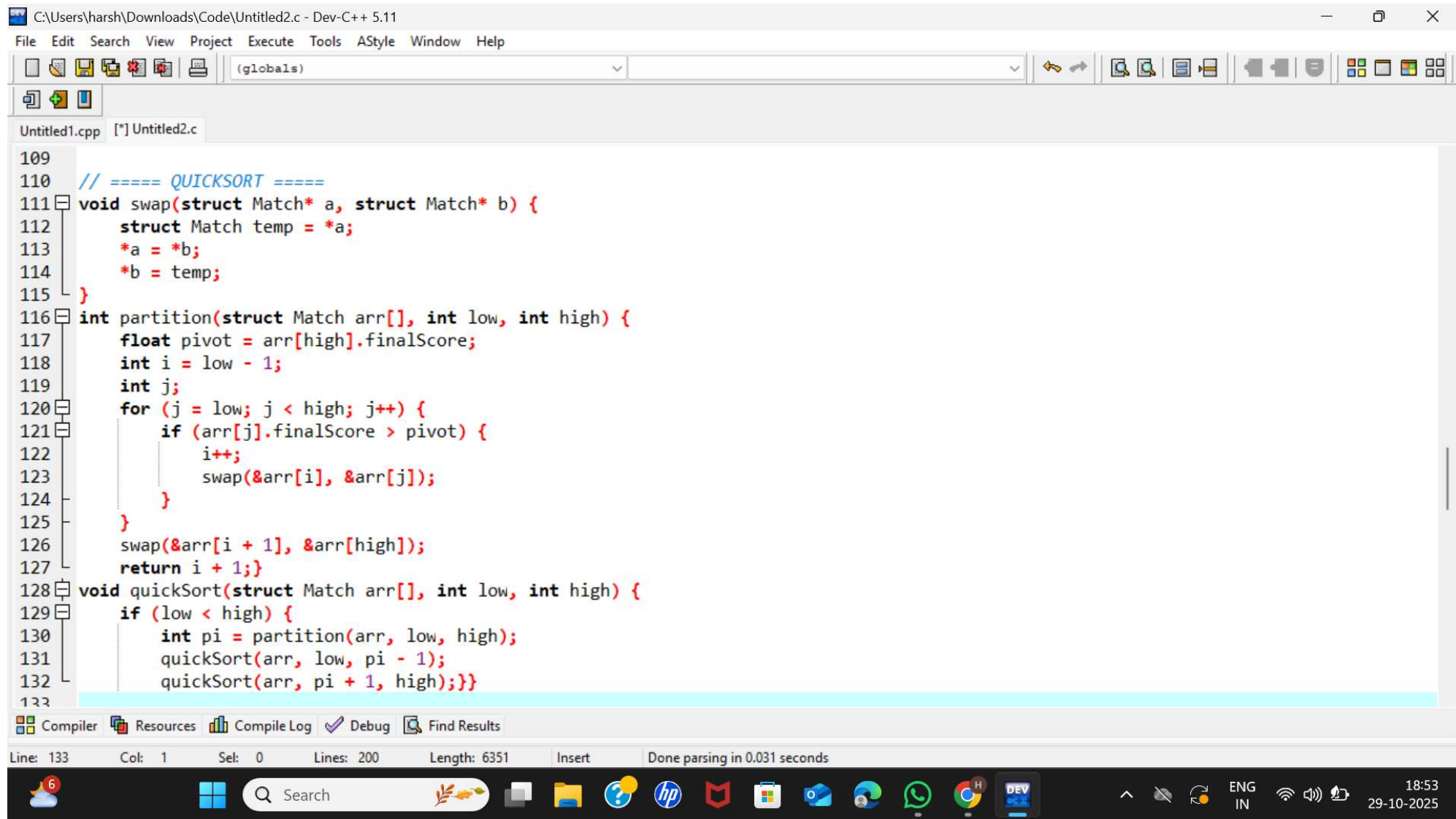
Untitled1.cpp Untitled2.c

```
91 // ===== SCORE CALCULATION =====
92 float calcScore(struct Donor d, struct Patient p) {
93     float s = 0;
94     if (!compatible(d.blood, p.blood)) return 0;
95     s += 40; // blood type match
96     if (strcmp(d.organ, p.need) == 0) s += 30;
97     {
98         int diff = abs(d.age - p.age);
99         if (diff <= 10) s += 15;
100        else if (diff <= 20) s += 10;
101        else if (diff <= 30) s += 5;
102    }
103    if (strcmp(d.health, "Excellent") == 0) s += 10;
104    else if (strcmp(d.health, "Good") == 0) s += 7;
105    else s += 4;
106    if (d.available) s += 5;
107    return s;
108 }
109
110 // ===== QUICKSORT =====
```

Compiler Resources Compile Log Debug Find Results

Line: 9 Col: 23 Sel: 0 Lines: 205 Length: 6365 Insert Done parsing in 0.031 seconds

6 Search hp M DEV 18:52 29-10-2025



C:\Users\harsh\Downloads\Code\Untitled2.c - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

(globals)

Untitled1.cpp [*] Untitled2.c

```
135 int main() {
136     struct Donor donors[MAX];
137     struct Patient patient;
138     struct Match matches[MAX];
139     int donorCount, count = 0, i;
140
141     printf("Enter number of donors (max %d): ", MAX);
142     scanf("%d", &donorCount);
143
144     for (i = 0; i < donorCount; i++) {
145         printf("\nEnter details for Donor %d:\n", i + 1);
146         printf("Name: ");
147         scanf("%s", donors[i].name);
148         printf("Blood Group: ");
149         scanf("%s", donors[i].blood);
150         printf("Organ Type: ");
151         scanf("%s", donors[i].organ);
152         printf("Age: ");
153         scanf("%d", &donors[i].age);
154         printf("Health (Excellent/Good/Fair): ");
155         scanf("%s", donors[i].health);
156         printf("Available (1=Yes, 0=No): ");
157         scanf("%d", &donors[i].available);
158         printf("City (Delhi/Mumbai/Bangalore/Chennai/Hyderabad/Kolkata): ");
159         scanf("%s", donors[i].location);
```

Compiler Resources Compile Log Debug Find Results

Line: 133 Col: 1 Sel: 0 Lines: 200 Length: 6351 Insert Done parsing in 0.031 seconds



Search



C:\Users\harsh\Downloads\Code\Untitled2.c - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

(globals)

Untitled1.cpp [*] Untitled2.c

```
169 printf("City (Delhi/Mumbai/Bangalore/Chennai/Hyderabad/Kolkata): ");
170 scanf("%s", patient.location);
171 for (i = 0; i < donorCount; i++) {
172     float s = calcScore(donors[i], patient);
173     if (s > 0) {
174         matches[count].score = s;
175         matches[count].distance = distance(donors[i].location, patient.location);
176         matches[count].finalScore = s - (matches[count].distance / 100);
177         strcpy(matches[count].name, donors[i].name);
178         count++;
179     }
180 }
181 if (count == 0) {
182     printf("\nNo compatible donors found.\n");
183     return 0;
184 }
185 quickSort(matches, 0, count - 1);
186 printf("\n=== Best Donor Matches ===\n");
187 for (i = 0; i < count; i++) {
188     printf("%d. %s | Score: %.1f | Distance: %.1f km | Final: %.1f\n",
189         i + 1, matches[i].name, matches[i].score,
190         matches[i].distance, matches[i].finalScore);
191 }
192 return 0;
193 }
194
```

Compiler Resources Compile Log Debug Find Results

Line: 195 Col: 1 Sel: 0 Lines: 195 Length: 6341 Insert Done parsing in 0.031 seconds

6 Search hp M 29-10-2025 18:56

Enter number of donors (max 10): 3

Enter details for Donor 1:

Name: Riya

Blood Group: A+

Organ Type: Kidney

Age: 25

Health (Excellent/Good/Fair): Excellent

Available (1=Yes, 0=No): 1

City (Delhi/Mumbai/Bangalore/Chennai/Hyderabad/Kolkata): Delhi

Enter details for Donor 2:

Name: Rahul

Blood Group: O+

Organ Type: Liver

Age: 40

Health (Excellent/Good/Fair): Good

Available (1=Yes, 0=No): 0

City (Delhi/Mumbai/Bangalore/Chennai/Hyderabad/Kolkata): Mumbai

```
C:\Users\harsh\Downloads\Cc x + v
Enter details for Donor 3:
Name: Sneha
Blood Group: A+
Organ Type: Kidney
Age: 32
Health (Excellent/Good/Fair): Fair
Available (1=Yes, 0=No): 1
City (Delhi/Mumbai/Bangalore/Chennai/Hyderabad/Kolkata): Hyderabad

Enter Patient Details:
Blood Group: A+
Needed Organ: Kidney
Age: 30
City (Delhi/Mumbai/Bangalore/Chennai/Hyderabad/Kolkata): Chennai

=== Best Donor Matches ===
1. Sneha | Score: 94.0 | Distance: 515.2 km | Final: 88.8
2. Riya | Score: 100.0 | Distance: 1755.8 km | Final: 82.4
3. Rahul | Score: 62.0 | Distance: 1033.1 km | Final: 51.7
```

CONCLUSION

This project provides an efficient, scalable, and life-saving digital infrastructure for blood and organ donation management. By leveraging optimized data structures and algorithms, it ensures quick response times, accurate matches, and real-time communication between donors and hospitals.

REFERENCES

- 1.DSA Textbooks & Notes
- 2.Research papers on Organ Donation Systems
- 3.MongoDB, React, Node.js Documentation
- 4.Mapbox/Leaflet API Documentation

PPT

SLIDES SHOULD SUMMARIZE:

- 1.Problem & Motivation
- 2.Objectives
- 3.Architecture
- 4.Algorithms Used
- 5.Flow Diagram
- 6.Screenshots & Results
- 7.Conclusion

**Thank you
very much!**